

CS202 Design Pattern Report: Observer Pattern

Decoupling State Changes, Push vs. Pull Notifications, and C++
Memory Management

Group 52

Nguyễn Đình Minh Huy (Student ID: 25125083)

Trần Gia Huy (Student ID: 25125084)

Ngày 20 tháng 8 năm 2026

Mục lục

1	Real-world Problem & Motivation	3
1.1	Background: Banking Event Notifications	3
1.2	Problem Statement	3
2	Naive Solution (Without Design Pattern)	4
2.1	Step-by-Step Execution of the Naive Solution	5
3	Analysis of Naive Solution Drawbacks	6
4	Introduction to the Observer Design Pattern	6
4.1	Definition and Intent	6
4.2	Key Structural Components	7
5	UML Class Diagrams	8
5.1	General GoF Observer Pattern Structure	8
5.2	Bank Account Notification Class Diagram	8
6	Pattern-Based Implementation in C++	17
6.1	Memory Management Considerations	9
6.2	Source Code Implementation	9
6.3	Execution Walkthrough & Dynamic Subscription Demo	13
6.3.1	Execution Trace Table	13
7	Evaluation of Trade-offs	14
7.1	Advantages	14
7.2	Disadvantages and Limitations	14
8	Modern Web and Mobile Engineering Applications	15
8.1	Web Development Applications	15
8.2	Mobile Development Applications	15

9 Interpreter and Mediator Integration	16
9.1 Combining Observer with the Interpreter Pattern	16
9.2 Combining Observer with the Mediator Pattern	16
10 Conclusion	16
References	17

1 Real-world Problem & Motivation

1.1 Background: Banking Event Notifications

In software development, maintaining consistency between core business entities and dependent logging or notification modules is a common requirement. In a banking system, a **BankAccount** class manages account state, balance updates, and transaction records.

When a customer performs an operation (such as a deposit, withdrawal, or failed overdraft attempt), multiple subsystems need to respond:

1. **User Interface (UI):** Updates the displayed balance and transaction list.
2. **Email Service:** Sends transaction receipts and security alerts to the user.
3. **Audit Logger:** Writes timestamped audit records for compliance.
4. **Fraud Detection Engine:** Evaluates transaction parameters to identify suspicious activities.
5. **Mobile Push Service:** Sends alert notifications to the user's mobile device.
6. **Analytics Service:** Records transaction data for operational metrics.

1.2 Problem Statement

The objective is to allow the **BankAccount** class to notify multiple interested services when a transaction occurs without coupling the account logic to specific service implementations. Furthermore, services must be able to subscribe or unsubscribe at runtime without requiring changes to existing classes or recompilation of the core banking logic.

2 Naive Solution (Without Design Pattern)

In a basic implementation, the `BankAccount` class maintains direct member pointers to each concrete service. When a transaction takes place, `BankAccount` calls specific methods on each service sequentially.

```
1 namespace naive {
2
3 class UIUpdateService {
4 public:
5     void updateUI(double amount, double balance) {
6         std::cout << "[UI] Updated. Transaction: $" << amount << ",
7         Balance: $" << balance << "\n";
8     }
9 };
10
11 class EmailService {
12 public:
13     void sendEmail(const std::string& accNum, double amount) {
14         std::cout << "[Email] Sent receipt for account " << accNum << "
15         . Amount: $" << amount << "\n";
16     }
17 };
18
19 class LoggerService {
20 public:
21     void logTransaction(const std::string& accNum, double amount) {
22         std::cout << "[Logger] Logged transaction for " << accNum << ".
23         Amount: $" << amount << "\n";
24     }
25 };
26
27 class FraudDetectionService {
28 public:
29     void detectFraud(const std::string& accNum, double amount) {
30         std::cout << "[Fraud AI] Checked account " << accNum << " for $"
31         << amount << ". Status: OK.\n";
32     }
33 };
34
35 class MobileAppService {
36 public:
37     void pushNotification(double amount) {
38         std::cout << "[Mobile App] Push alert sent. Amount: $" <<
39         amount << "\n";
40     }
41 };
42
43 class AnalyticsService {
44 public:
45     void trackEvent(const std::string& eventType, double amount) {
46         std::cout << "[Analytics] Event ' ' << eventType << ' ' logged
47         for $" << amount << "\n";
48     }
49 };
50
51 // BankAccount directly coupled to concrete service classes
52 class BankAccount {
```

```

47 private:
48     double balance;
49     std::string accountNumber;
50
51     // Direct dependencies on 6 concrete classes
52     UIUpdateService* uiService;
53     EmailService* emailService;
54     LoggerService* loggerService;
55     FraudDetectionService* fraudService;
56     MobileAppService* mobileService;
57     AnalyticsService* analyticsService;
58
59 public:
60     BankAccount(std::string accNum,
61                 UIUpdateService* ui, EmailService* email,
62                 LoggerService* logger, FraudDetectionService* fraud,
63                 MobileAppService* mobile, AnalyticsService* analytics)
64         : balance(0.0), accountNumber(accNum),
65           uiService(ui), emailService(email), loggerService(logger),
66           fraudService(fraud), mobileService(mobile), analyticsService(
67 analytics) {}
68
69     void deposit(double amount) {
70         balance += amount;
71
72         // Explicit dispatch calling each service method directly
73         uiService->updateUI(amount, balance);
74         emailService->sendEmail(accountNumber, amount);
75         loggerService->logTransaction(accountNumber, amount);
76         fraudService->detectFraud(accountNumber, amount);
77         mobileService->pushNotification(amount);
78         analyticsService->trackEvent("deposit", amount);
79     }
80 };
81 } // namespace naive

```

Listing 1: Naive Bank Account with Direct Service Coupling (naive.cpp)

2.1 Step-by-Step Execution of the Naive Solution

To illustrate the behavior of the naive implementation, consider depositing \$500.00:

1. **Initialization:** The client instantiates all 6 concrete service objects and passes their pointers to the `BankAccount` constructor.
2. **State Update:** Invoking `bankAccount.deposit(500.0)` increments the internal `balance` attribute by \$500.00.
3. **Sequential Dispatch:** The `deposit()` method executes the following calls:
 - `uiService->updateUI(500.0, 500.0)`
 - `emailService->sendEmail("ACC-101", 500.0)`
 - `loggerService->logTransaction("ACC-101", 500.0)`
 - `fraudService->detectFraud("ACC-101", 500.0)`

- `mobileService->pushNotification(500.0)`
- `analyticsService->trackEvent("deposit", 500.0)`

3 Analysis of Naive Solution Drawbacks

The naive approach presents several design limitations:

Issue	Cause in Code	Effect on System
Open-Closed Principle (OCP) Violation	Adding a service requires modifying the <code>BankAccount</code> member variables, constructor, and <code>deposit()</code> method.	Existing banking code must be altered and retested whenever notification requirements change.
Single Responsibility Principle (SRP) Violation	<code>BankAccount</code> manages balance state while also coordinating logging, UI updates, and email dispatching.	Increases class complexity and couples financial logic to external I/O operations.
Tight Coupling	<code>BankAccount</code> references concrete classes and depends on their individual method signatures.	Signature changes in service classes require corresponding edits in <code>BankAccount</code> .
Static Subscriptions	Service pointers are assigned at construction time.	Services cannot be enabled, disabled, or substituted dynamically during execution.
Testing Overhead	Testing <code>BankAccount</code> requires instantiating or managing instances of all 6 concrete services.	Increases testing dependencies and makes isolated unit testing more difficult.

Bảng 1: Design Issues in the Naive Implementation

4 Introduction to the Observer Design Pattern

4.1 Definition and Intent

According to the Gang of Four (GoF), the **Observer Pattern** is defined as:

"Define a one-to-many dependency between objects so that when one object changes state, all its dependents are notified and updated automatically."

The pattern separates the object that generates events (the **Subject**) from the objects that handle those events (the **Observers**) by using an abstract interface.

4.2 Key Structural Components

- **Subject (BankAccount):** Maintains a collection of observer pointers and provides subscription methods:

`attach(Observer)` Registers a new subscriber.

`detach(Observer)` Removes a registered subscriber.

`notify(...)` Iterates through subscribers and calls their update method.

- **Observer Interface (AccountObserver):** Declares the update interface:

```
virtual void onTransaction(const std::string& type, double amount, double balance)
```

- **Concrete Observers (UIObserver, EmailObserver, LoggerObserver, etc.):** Implement the interface to execute their specific response logic.

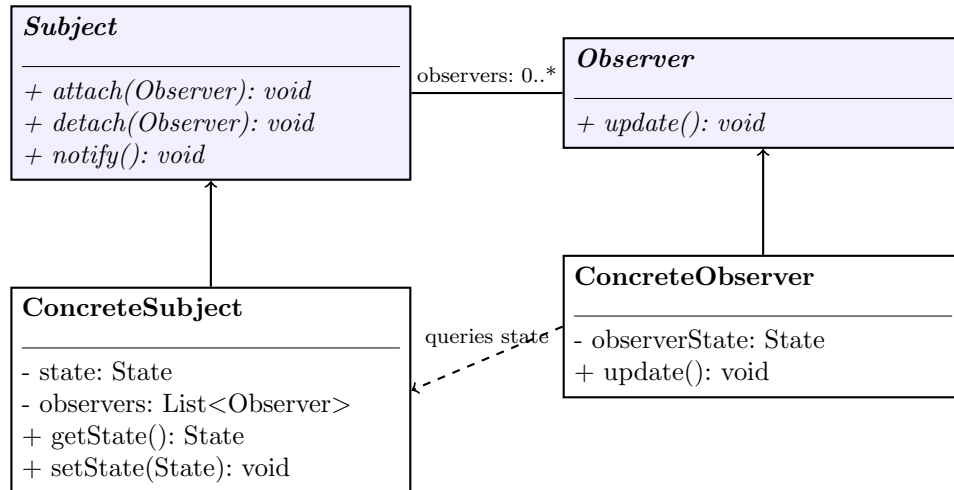
- **Push vs. Pull Notification Models:**

- **Push Model:** The Subject sends state details (`type`, `amount`, `balance`) directly as method arguments during the notification call.
- **Pull Model:** The Subject passes a reference to itself, and observers query the specific state attributes they need.

5 UML Class Diagrams

5.1 General GoF Observer Pattern Structure

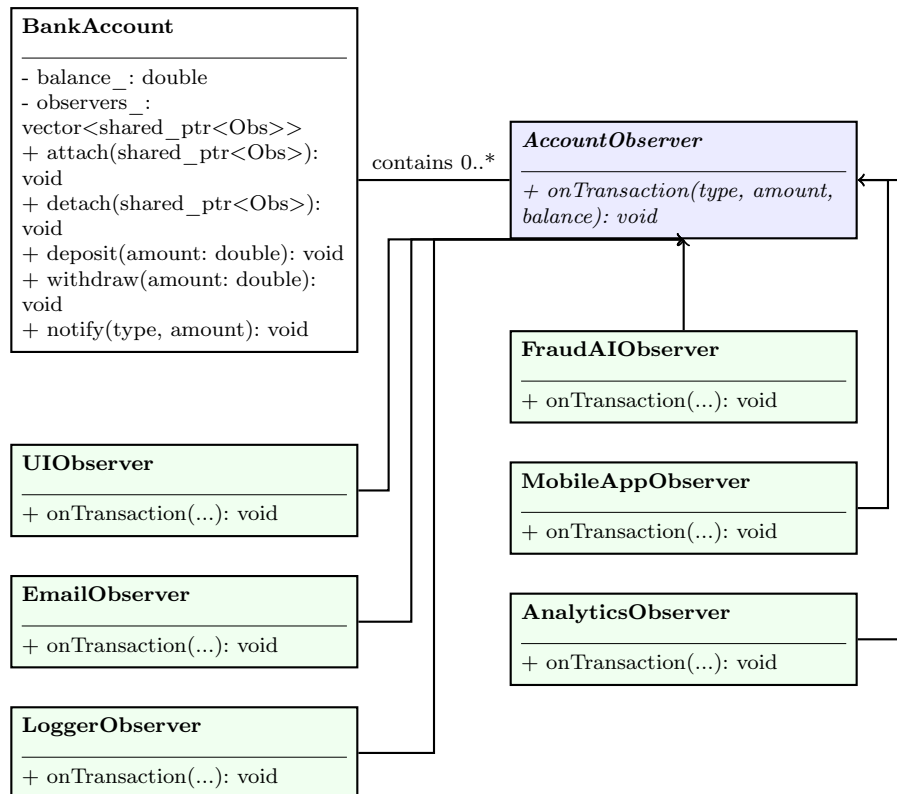
The general Observer pattern structure connects the **Subject** to the abstract **Observer** interface rather than concrete subscriber classes.



Hình 1: General GoF Observer Pattern UML Structure

5.2 Bank Account Notification Class Diagram

In the bank account system, **BankAccount** acts as the **Subject**, storing a collection of `std::shared_ptr<AccountObserver>` instances. All 6 concrete notification classes implement the **AccountObserver** interface.



Hình 2: Bank Account Observer Pattern Class Hierarchy

6 Pattern-Based Implementation in C++17

6.1 Memory Management Considerations

In C++, managing observer lifetimes requires addressing potential reference issues:

- **Raw Pointers:** If an observer is deleted while remaining in the subject's list, subsequent notifications can result in dangling pointer access.
- **Smart Pointers (`std::shared_ptr` and `std::weak_ptr`):**
 - **`std::shared_ptr`:** Keeps the observer instance valid while it is held in the subject's subscription list.
 - **`std::weak_ptr`:** Allows the subject to observe whether the subscriber still exists via `lock()` before invoking callbacks, preventing memory leaks when observers are destroyed externally.

6.2 Source Code Implementation

```

1 #include <iostream>
2 #include <memory>
3 #include <vector>
4 #include <string>
5 #include <algorithm>
6
7 // Observer Interface
8 class AccountObserver {

```

```

9 public:
10     virtual ~AccountObserver() = default;
11     virtual void onTransaction(const std::string& type, double amount,
12                               double currentBalance) = 0;
13 };
14
15 // Subject Class
16 class BankAccount {
17 public:
18     BankAccount(double initialBalance = 0.0) : balance_(initialBalance)
19     {}
20
21     void attach(std::shared_ptr<AccountObserver> observer) {
22         observers_.push_back(observer);
23     }
24
25     void detach(std::shared_ptr<AccountObserver> observer) {
26         observers_.erase(
27             std::remove(observers_.begin(), observers_.end(), observer)
28             ,
29             observers_.end()
30         );
31     }
32
33     void notify(const std::string& type, double amount) {
34         for (const auto& observer : observers_) {
35             observer->onTransaction(type, amount, balance_);
36         }
37     }
38
39     void deposit(double amount) {
40         balance_ += amount;
41         std::cout << "[BankAccount] Deposited $" << amount << ". New
42         balance: $" << balance_ << "\n";
43         notify("Deposit", amount);
44     }
45
46     void withdraw(double amount) {
47         if (amount > balance_) {
48             std::cout << "[BankAccount] Rejected: Insufficient funds
49             for withdrawal of $" << amount << "\n";
50             notify("FailedWithdrawal", amount);
51         } else {
52             balance_ -= amount;
53             std::cout << "[BankAccount] Withdrew $" << amount << ". New
54             balance: $" << balance_ << "\n";
55             notify("Withdrawal", amount);
56         }
57     }
58
59     double getBalance() const { return balance_; }
60
61 private:
62     double balance_;
63     std::vector<std::shared_ptr<AccountObserver>> observers_;
64 };

```

Listing 2: Observer Interface and BankAccount Subject Implementation (pattern.cpp)

```

1 class UIObserver : public AccountObserver {
2 public:
3     void onTransaction(const std::string& type, double amount, double
currentBalance) override {
4         (void)type; (void)amount;
5         std::cout << "[UI Observer] Balance display updated to $" <<
currentBalance << "\n";
6     }
7 };
8
9 class EmailObserver : public AccountObserver {
10 public:
11     void onTransaction(const std::string& type, double amount, double
currentBalance) override {
12         (void)currentBalance;
13         if (type == "Deposit") {
14             std::cout << "[Email Observer] Alert sent: A deposit of $"
<< amount << " was made.\n";
15         } else if (type == "Withdrawal") {
16             std::cout << "[Email Observer] Alert sent: A withdrawal of
$" << amount << " was made.\n";
17         } else if (type == "FailedWithdrawal") {
18             std::cout << "[Email Observer] Security Alert: Withdrawal
of $" << amount << " failed (Insufficient funds).\n";
19         }
20     }
21 };
22
23 class LoggerObserver : public AccountObserver {
24 public:
25     void onTransaction(const std::string& type, double amount, double
currentBalance) override {
26         std::cout << "[Logger Observer] Transaction logged: " << type
<< " of $" << amount << ", new balance: $" <<
currentBalance << "\n";
27     }
28 };
29
30
31 class FraudAIObserver : public AccountObserver {
32 public:
33     void onTransaction(const std::string& type, double amount, double
currentBalance) override {
34         (void)currentBalance;
35         if (amount > 10000.0) {
36             std::cout << "[Fraud AI] WARNING: Suspicious activity
detected: " << type << " of $" << amount << "\n";
37         } else {
38             std::cout << "[Fraud AI] Scan complete: " << type << " of $
" << amount << " marked OK.\n";
39         }
40     }
41 };
42
43 class MobileAppObserver : public AccountObserver {
44 public:
45     void onTransaction(const std::string& type, double amount, double
currentBalance) override {
46         (void)currentBalance;

```

```

47         std::cout << "[Mobile App] Push notification: " << type << " of
48         $" << amount << " processed.\n";
49     };
50
51     class AnalyticsObserver : public AccountObserver {
52     public:
53         void onTransaction(const std::string& type, double amount, double
54         currentBalance) override {
55             (void)currentBalance;
56             std::cout << "[Analytics Observer] Event registered: Type=" <<
57             type << ", Volume=$" << amount << "\n";
58         }
59     };

```

Listing 3: Concrete Observer Implementations

6.3 Execution Walkthrough & Dynamic Subscription Demo

The execution driver demonstrates attaching all 6 observers, running a deposit, detaching the Email and Mobile App observers at runtime, and verifying the subsequent withdrawal output.

```
1 void runPatternDemo() {
2     BankAccount account(1000.0);
3
4     auto ui = std::make_shared<UIObserver>();
5     auto email = std::make_shared<EmailObserver>();
6     auto logger = std::make_shared<LoggerObserver>();
7     auto fraud = std::make_shared<FraudAIObserver>();
8     auto mobile = std::make_shared<MobileAppObserver>();
9     auto analytics = std::make_shared<AnalyticsObserver>();
10
11     // 1. Attach all 6 observers
12     account.attach(ui); account.attach(email); account.attach(logger);
13     account.attach(fraud); account.attach(mobile); account.attach(
14     analytics);
15
16     // 2. Perform Deposit
17     std::cout << "\n--- TEST 1: Deposit $500.00 (All 6 Observers Active
18     ) ---\n";
19     account.deposit(500.0);
20
21     // 3. Dynamic Detachment
22     std::cout << "\n--- TEST 2: Detaching Email & Mobile Observers ---\n";
23     account.detach(email);
24     account.detach(mobile);
25
26     // 4. Perform Subsequent Withdrawal
27     std::cout << "\n--- TEST 3: Withdrawal $200.00 (Email & Mobile
28     Inactive) ---\n";
29     account.withdraw(200.0);
30 }
```

Listing 4: Observer Pattern Execution Demo (runPatternDemo)

6.3.1 Execution Trace Table

Step	Operation	Active Observers	Dispatched Output
1	Init (Bal=\$1000)	UI, Email, Logger, Fraud, Mobile, Analytics	Initial state initialized with 6 subscribers attached.
2	deposit(500.0)	All 6 Observers	UI updates balance to \$1500; Email sends receipt; Logger records entry; Fraud AI checks amount; Mobile sends alert; Analytics logs event.
3	detach(email, mobile)	UI, Logger, Fraud, Analytics	Email and Mobile App observers removed from list.
4	withdraw(200.0)	UI, Logger, Fraud, Analytics	UI updates balance to \$1300; Logger records \$200 withdrawal; Fraud AI checks amount; Analytics logs event. Email and Mobile receive no events.

Bảng 2: Dynamic Subscription and Event Broadcast Trace

7 Evaluation of Trade-offs

7.1 Advantages

- **Open-Closed Principle (OCP):** New observer classes (such as tax calculators or external reporting tools) can be added without modifying `BankAccount`.
- **Loose Coupling:** `BankAccount` depends only on the abstract `AccountObserver` interface, separating transaction logic from output mechanisms.
- **Runtime Configuration:** Observers can attach or detach dynamically based on application requirements.
- **Broadcast Distribution:** A single state update automatically notifies all registered observers.

7.2 Disadvantages and Limitations

- **Lapsed Listener Issues:** If observers are destroyed without detaching, the subject may retain inactive references, causing resource leaks or undefined behavior when raw pointers are used.
- **Unordered Notification:** Observers are invoked in list order, so individual observers must not assume specific execution ordering relative to other observers.
- **Cascading Updates:** If an observer calls a method on the subject that triggers another notification, it can lead to unintended recursive updates.

8 Modern Web and Mobile Engineering Applications

8.1 Web Development Applications

- **DOM Event Handling:** In JavaScript, `element.addEventListener('click', callback)` registers callbacks to handle UI events.
- **State Management Libraries:** In frameworks using Redux, Zustand, or Pinia, view components subscribe to state stores and re-render when state changes.
- **WebSocket Event Distribution:** In Socket.io and similar protocols, clients subscribe to server channels to receive real-time messages.

8.2 Mobile Development Applications

- **Android Jetpack LiveData and StateFlow:** Views observe data streams emitted by ViewModels and update UI elements when values change.
- **iOS SwiftUI and Combine:** Views bind to `@Published` variables in observable models and refresh automatically upon state changes.

9 Interpreter and Mediator Integration

9.1 Combining Observer with the Interpreter Pattern

The Observer and Interpreter patterns can be combined in rule-processing engines:

- **Context as Subject:** In an expression evaluator, the symbol table (`Context`) serves as an observable Subject.
- **Expression Trees as Observers:** Root nodes of Abstract Syntax Trees register as observers for specific variable keys.
- **Automatic Re-Evaluation:** When variable values update in the Context, subscribed syntax trees re-evaluate automatically without requiring polling.

9.2 Combining Observer with the Mediator Pattern

In systems with many interacting components, combining the Observer and Mediator patterns provides an event aggregator structure:

- **Central Event Hub:** Components register as observers with a central `EventMediator` rather than subscribing directly to individual peers.
- **Decoupled Event Routing:** When a component generates an event, the Mediator routes it to the relevant subscriber components.
- **Topology Simplification:** Reduces point-to-point connections from $O(N^2)$ to an $O(N)$ star topology while supporting broadcast messaging.

10 Conclusion

The Observer Design Pattern provides a structured way to decouple state-holding subjects from dependent notification and logging modules. By using an abstract interface and maintaining dynamic subscriber collections, the pattern supports the Open-Closed and Single Responsibility Principles.

In C++17, utilizing smart pointers (`std::shared_ptr` and `std::weak_ptr`) ensures safe object lifetime management and prevents dangling references. When integrated with the Interpreter and Mediator patterns, the Observer pattern supports reactive rule engines and centralized event-driven architectures.

References

- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.
- Meyers, S. (2014). *Effective Modern C++: 42 Specific Ways to Improve Your Use of C++11 and C++14*. O'Reilly Media.
- Fowler, M. (2002). *Patterns of Enterprise Application Architecture*. Addison-Wesley.
- Refactoring.Guru. *Design Patterns: Observer Pattern*. <https://refactoring.guru/design-patterns/observer>